

MÓDULO 8: SEGURIDAD

IDENTIFICATION AND AUTHENTICATION FAILURES

OWASP TOP 10

BIG school

Identification and Authentication Failures

Índice

- Introducción: la puerta de entrada de todo sistema
- ¿Qué significa "Identification and Authentication Failures"?
- Ejemplos comunes de fallos de autenticación
- ¿Qué puede hacer un atacante si explota esto?
- ¿Cómo detectar fallos de autenticación?
- ¿Cómo prevenirlo?
- ¿Cómo mitigar si ya estás afectado?
- Checklist rápido
- Conclusiones



Identification and Authentication Failures

BIG school

Introducción: la puerta de entrada de todo sistema

- Muchos ataques comienzan con un inicio de sesión débil.
- Esta vulnerabilidad aparece cuando la autenticación o la gestión de identidad están mal implementadas.
- Incluye cómo se crean, almacenan y validan contraseñas, y cómo se gestionan las sesiones.

¿Qué significa "Identification and Authentication Failures"?

Identificación: proceso por el cual el sistema sabe quiénes eres (ej. correo o usuario).

Autenticación: confirma que realmente eres quien dices ser (contraseña, token o biometría).

Los fallos aparecen cuando:

- Permiten acceder sin credenciales válidas.
- Se reutilizan sesiones de otros usuarios.
- Se evita o elude la autenticación.
- Impacta directamente en la confidencialidad y el control de acceso.

Ejemplos comunes de fallos de autenticación

- Contraseñas débiles o por defecto: admin:admin, 123456.
- Políticas inseguras: sin límites de intentos o complejidad mínima.
- Almacenamiento inseguro: contraseñas en texto plano o con hash débil (MD5, SHA-1).
- Sesiones mal gestionadas: tokens sin expiración o reusados.
- MFA ausente o mal implementada: omisión de doble factor en cuentas críticas.
- Recuperación insegura: enlaces que no expiran o revelan existencia de cuentas.
- JWT o API Keys vulnerables: sin expiración o con firmas débiles.

¿Qué puede hacer un atacante si explota esto?

- Acceso a cuentas ajenas y robo de datos.
- Escalada de privilegios a cuentas administrativas.
- Secuestro de sesiones y movimiento lateral en el sistema.
- Acceso a paneles o APIs con credenciales por defecto.
- Filtrado de información sensible.

¿Cómo detectar fallos de autenticación?

- Revisión de código: buscar contraseñas hardcodeadas, algoritmos inseguros o flujos mal definidos.
- Pentesting: probar fuerza bruta, reutilización de sesiones, control de expiración.
- Revisión de configuración: validar políticas de contraseñas, MFA y expiración de sesiones.
- Escaneos automáticos: OWASP ZAP, Nikto o SAST/DAST para endpoints débiles.
- Monitoreo de actividad: detectar patrones de login anormales o intentos masivos.
- Un sistema seguro debe alertar ante intentos sospechosos.

¿Cómo prevenirlo?

- **Contraseñas seguras:** mínimo 12 caracteres o passphrases, verificar contraseñas vulnerables.
- **Almacenamiento seguro:** usar bcrypt, scrypt o Argon2 con sal única.
- **Protección contra fuerza bruta:** limitar intentos y aplicar bloqueos temporales.
- **Gestión de sesiones:** tokens únicos, expiración razonable, flags HttpOnly y Secure.
- **Autenticación multifactor:** activar MFA para usuarios y administradores.
- **Recuperación de contraseñas:** enlaces únicos, caducidad corta e invalidación tras uso.
- **Seguridad en APIs/JWT:** tokens con expiración, firma fuerte y revocación rápida.
- **Registro y monitoreo:** log de accesos, cambios y fallos repetidos.

¿Cómo mitigar si ya estás afectado?

- Revisar logs de accesos y detectar anomalías.
- Forzar cambio de contraseñas.
- Invalidar todas las sesiones activas.
- Activar MFA obligatoria.
- Auditar el código y corregir los flujos inseguros.
- Informar a los usuarios afectados y reforzar el monitoreo.
- Documentar el incidente y actualizar políticas.

Checklist rápido

1. ¿Quedan contraseñas por defecto?
2. ¿Las contraseñas están encriptadas con bcrypt o Argon2?
3. ¿Sesiones con expiración y revocación?
4. ¿MFA activo en usuarios críticos?
5. ¿Tokens JWT firmados y con expiración corta?
6. ¿Recuperación de contraseñas segura?

Conclusiones

1. La autenticación es la primera línea de defensa.
2. Cada formulario o token que implementas es una puerta al sistema.
3. No basta con que "funcione": debe ser confiable y resistente.
4. Usa frameworks seguros, integra MFA y sigue las guías OWASP.
5. Proteger usuarios reales es proteger la reputación de tu software.

Aplicar autenticación segura te convierte no solo en buen desarrollador, sino en uno responsable.